

DUBAI FUTURE FOUNDATION · 10X PHASE



City Terminal

مدينة ترمينال — Smart Off-Airport Check-In Platform

End-to-end digital journey for passengers, drivers, and operations teams — from booking verification and passport OCR to live vehicle tracking, terminal handover, and boarding-ready confirmation.

Project Brief · Confidential · May 2026

Prepared for: Dubai Future Foundation (DFF) — 10X Innovation Track

Initiative alignment: Dubai 10X · Future of mobility & airport experience

Executive Summary

City Terminal is a multi-surface platform built under the **Dubai Future Foundation (DFF) 10X phase**, designed to extend the airport check-in experience beyond the terminal — enabling passengers to complete identity verification, schedule curb-side pickup slots, track dedicated vehicles in real time, and receive a digital boarding pass–style confirmation before reaching Dubai International Airport (DXB).

Vision

Reduce terminal congestion and waiting time by shifting verification and logistics to a coordinated “City Terminal” network with live operations visibility.

Delivery Status

Working prototype across passenger mobile app, driver app, operations admin dashboard, Node.js API, PostgreSQL (Supabase), and bilingual AI chat service (JAIS + OpenAI).

Key outcomes demonstrated

- **Passenger app (React Native / Expo)** — 20+ screens including a **two-stage QR model**: *Luggage / Baggage Receipt QR* unlocks when the driver verifies pickup; *Digital Boarding Pass QR* unlocks only after the vehicle reaches the airport terminal.
- **Driver app** — Supabase-authenticated portal: accept bookings, trip status (Accepted → Picked up → Verified → Done), real-time dispatch sync.
- **Admin operations dashboard (React + Vite)** — Live stats, dispatch map, passenger search, analytics, terminal arrival alerts with audio cue.
- **Backend API (Express 5 + Socket.IO)** — Bookings, passport scan, slots, OTP pickup proof, driver tasks, confirmation flows.
- **AI service (FastAPI)** — Bilingual chat (English / Arabic), RAG over booking data, JAIS LLM integration, passport OCR endpoint.

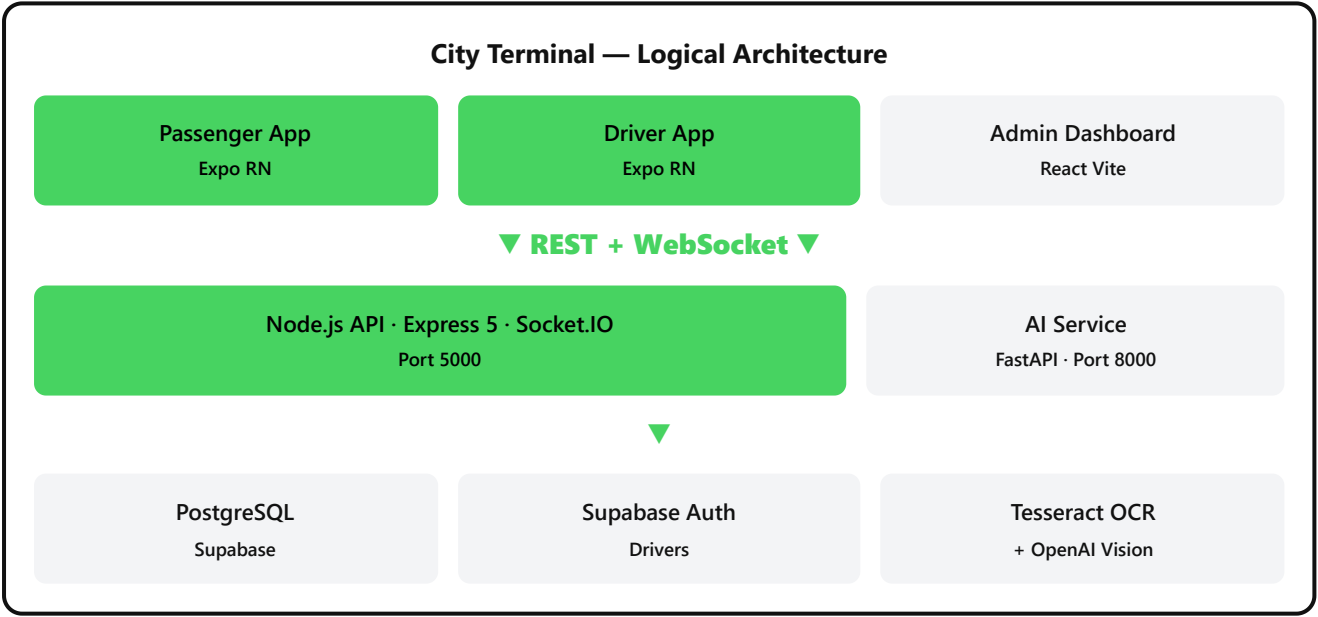
Table of Contents

1. Problem statement & solution
2. System architecture
3. Platform modules (detailed)
4. Passenger journey flow
5. Technology stack
6. Implemented capabilities
7. AI & intelligence layer
8. Data, security & integrations
9. Future roadmap (10X scale-up)
10. API surface summary

Problem Statement & Solution

Challenge	City Terminal response
Long queues and repeated document checks at airport	Pre-terminal passport capture with OCR + booking cross-check before travel day
Uncoordinated pickup from hotels / residences	Time-slot booking, AI-assisted pickup recommendation, assigned vehicle & driver ID
No visibility for passenger or control tower	Socket.IO live location + status; admin dispatch map; passenger Live Tracking screen
Manual driver-passenger handover	Pickup OTP, proof QR generation, driver "Verified / Done" workflow on dedicated driver app
Language & support barriers	Arabic / English i18n across app; JAIS-powered bilingual AI chat with booking context

System Architecture



Repository structure

Folder	Role
mobile-app/	Passenger-facing Expo application
driver-app/	Driver login, task list, trip console
admin-dashboard/	Operations control tower (web)

backend/	Core REST API, real-time events, passport OCR
ai-service/	Bilingual chatbot, recommendations, OCR helper
frontend/	Shared UI components / marketing assets (React)

Platform Modules — Detailed

1. Passenger Mobile Application

React Native (Expo 54) with React Navigation, i18next (EN/AR RTL), Supabase session storage, and Careem-inspired green/black design system.

Screen / Module	Function
Splash & Language Select	DFF-branded entry; bilingual UX from first interaction
Auth	Email validation & passenger session
Booking Entry	Verify PNR/booking ID + airline against database
Passport Scan & Verification	Camera/upload → Tesseract OCR (server) + MRZ parse → name, passport #, DOB, nationality with edit & save
Time Slot / Slot Booking	Available slots filtered before flight departure; AI recommend slot/pickup
Location Pick & Booking Confirm	Pickup address, vehicle assignment preview
Live Tracking	Real-time van map + Socket.IO; Luggage QR appears here the moment driver taps Verify (status barcode_issued)
User Profile / Barcode	Baggage Receipt QR (type: baggage_receipt) — bag status "With driver"; same ID for luggage handover
Confirmation	Digital Boarding Pass QR (type: digital_boarding_pass) — only when at_airport / terminal reached; bag status "At terminal"
Chat Support	Streaming AI assistant with booking context
User Profile & Driver Trip (embedded)	Trip persistence; optional driver-facing flows in same codebase

2. Driver Application

- Supabase email/password authentication with Driver ID metadata
- Task inbox: pending bookings assigned by vehicle ID or driver ID (e.g. DR-105 / CT-vehicle)
- **Accept booking** — binds driver to trip via POST /api/otp/accept-booking
- Trip console: Accept → En route → **Verified** (passenger instantly receives **Luggage QR**) → En route to airport → **Done** (passenger receives **Boarding Pass QR**)
- Socket refresh when new bookings arrive on dispatch channel

3. Operations Admin Dashboard

- JWT-secured login (/api/admin/auth)
- Live KPIs: today's bookings, active vehicles, available slots
- Dispatch map with normalized coordinates; "arrived at terminal" visual state
- Passenger search, hourly analytics, peak-hour charts
- Real-time Socket.IO: location_update, status_update, terminal sound on arrival

4. Backend API Service

Route group	Capabilities
/api/bookings	Verify booking, fetch details, update passport fields, trip status
/api/passport/scan	Image upload (base64) → OpenAI Vision (optional) → Tesseract fallback → MRZ normalization → booking name match
/api/slots	Available slots, book slot, confirm pickup OTP, proof QR, slot-by-booking
/api/otp	Driver tasks, accept booking, pickup verification flows
/api/confirmation	Airport arrival & boarding confirmation state
/api/ai	Pickup & slot recommendation proxies
/api/admin/*	Auth + operations stats, vehicles, passengers, analytics
Socket.IO	Rooms: booking:{id}, dispatch — live location & status fan-out

5. AI Microservice (FastAPI)

- **JAIS 13B** (Inception / UAE bilingual model) for Arabic; OpenAI for English — auto language detect
- Streaming POST /chat with booking-aware context from Supabase REST
- Passport OCR via Tesseract (PIL pipeline) as auxiliary endpoint
- System prompts position City Terminal as DFF smart airport check-in service

Dual QR Journey — Luggage vs Boarding Pass

City Terminal deliberately separates identity documents into **two QR phases** so passengers always know whether their bag is with the driver or already at the airport terminal — and so airport staff see the correct credential at each gate.

PHASE 1 · AFTER DRIVER VERIFICATION Luggage / Baggage Receipt QR • Triggered when driver confirms pickup (vehicle_verified = true, status barcode_issued) • Backend issues JSON payload: type: baggage_receipt, documentType: Baggage Receipt • Socket.IO pushes status_update with vehicleVerified: true — no app refresh needed	PHASE 2 · AFTER AIRPORT TERMINAL REACHED Digital Boarding Pass QR • Triggered when trip status becomes at_airport (reached_airport = true) • Backend re-issues payload: type: digital_boarding_pass, documentType: Digital Boarding Pass • Passenger navigates to Confirmation screen (or Home shortcut if already at airport) • Full-screen boarding-style card with scannable QR via react-native-qrcode-svg
---	---

- Passenger sees QR on **Live Tracking, My Profile**, and **Barcode** screens
- Encodes booking ID, vehicle, driver, flight, destination — bag status: *With driver*

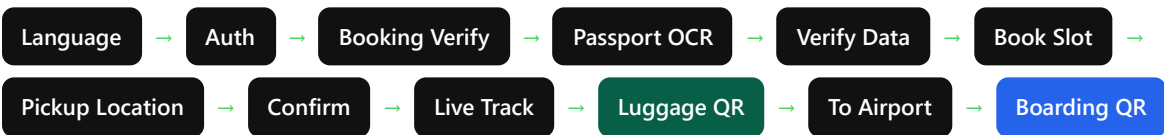
Driver taps “Verified” → Passenger gets Luggage QR

- Same booking reference upgrades visually — bag status: *At terminal*

Driver marks airport arrival → Boarding Pass QR unlocks

Trigger	Who acts	Passenger sees	QR type
Pickup verified	Driver app — Verify / OTP confirm	Luggage QR on Live Tracking & Profile	baggage_receipt
Terminal reached	Driver app — Done / at airport status	Boarding Pass on Confirmation screen	digital_boarding_pass

Passenger Journey Flow



Trip state is persisted locally (passengerTripStorage) so Home navigation resumes Live Tracking or Confirmation appropriately after app restart.

Technology Stack

Layer	Technologies
Mobile	React Native 0.81, Expo 54, React Navigation 7, i18next, expo-camera, react-native-maps, Socket.IO client, tesseract.js (web fallback)
Driver	Expo, Supabase JS SDK, AsyncStorage session
Admin Web	React 19, Vite 7, React Router 7, Socket.IO client
API	Node.js, Express 5, Socket.IO 4, pg (PostgreSQL), JWT, tesseract.js, OpenAI SDK
AI	Python FastAPI, LangChain RAG patterns, JAIS via API, OpenAI GPT-4o-mini, pytesseract
Data	PostgreSQL on Supabase — bookings, slots, slot_bookings, drivers, vehicle_tracking

Implemented Capabilities (Current Build)

- Tesseract OCR
- MRZ parsing
- Nationality display
- Real-time tracking
- Pickup OTP
- Proof QR
- Bilingual UI
- JAIS chatbot
- Admin dispatch
- Driver accept flow
- Slot capacity
- Flight-time filter

Passport & identity (current)

- Server-side **Tesseract OCR** with optional **OpenAI Vision** enrichment (PASSPORT_SCAN_METHOD=auto)
- Extracts: full name, passport number, date of birth, nationality, expiry, gender, confidence score
- Cross-validation against booking record in PostgreSQL
- Passenger can correct fields on Verification screen; persisted via PUT /api/bookings/:id/passport
- Web client fallback: device-side tesseract.js when API unreachable

Operations & logistics (current)

- Slot inventory with capacity and booking counts; demo fallback if DB empty
- Vehicle assignment, driver name, pickup OTP stored on slot_bookings
- Driver task API lists pending / active trips per driver or vehicle ID
- **Dual QR gate:** Luggage QR only after driver verify; Boarding Pass QR only after airport terminal — enforced in API (buildBarcodePayload) and UI (vehicleVerified / isBoardingPass checks)

AI & Intelligence Layer

Feature	Implementation
Conversational support	Streaming chat; language auto-detect; booking-aware answers (QR, vehicle, ETA, flight)
Pickup recommendation	POST /api/ai/recommend-pickup — factors location, traffic patterns (extensible)
Slot recommendation	POST /api/ai/recommend-slot — aligns slot with flight departure window
Document OCR	Dual pipeline: cloud vision + on-prem Tesseract for resilience

Future Roadmap — 10X Scale-Up

The current prototype establishes the full journey skeleton. The following integrations are planned for production alignment with UAE government digital services and airline operations.

1 UAE Passport Verification API (ICP / Official Channels)

Replace and augment OCR-only verification with **authoritative identity validation** through the UAE federal digital identity ecosystem (e.g. ICP Emirates ID / passport validation services). Flow: capture MRZ → submit tokenized request → receive verified status, nationality, and document authenticity without storing raw biometrics on device. Reduces fraud and manual correction on Verification screen.

2 Smart Luggage — Overweight Detection via Camera

Passenger photographs checked bag before pickup. **Computer vision model** estimates dimensions/volume class and flags likely **overweight or oversized** items against airline allowance rules (weight tiers per airline_code in booking). Early warning: “Estimated 24 kg — Emirates allowance 23 kg” with option to repack or pay excess before vehicle dispatch. Reduces surprises at curb and airport belt.

3 Flight Delay & Disruption Notifications

Integrate **flight status feeds** (airline APIs / OAG-style data). On delay or gate change: push notification + in-app banner; auto-suggest **rescheduled pickup slot**; notify assigned driver and operations dashboard. Socket event flight_update propagates to passenger Live Tracking and dispatch map.

4 DXB & Airline System Integration

Boarding pass sync, bag tag pre-registration, and terminal-specific routing (T1/T2/T3) based on airline master data.

5 Predictive Dispatch & Fleet Optimization

ML on historical slot_bookings + traffic: dynamic vehicle allocation, ETA confidence intervals, SLA dashboards for DFF operations.

UAE Passport API

Luggage CV

Push notifications

Flight status API

Apple / Google Wallet pass

ePayments for excess baggage

Data, Security & Compliance Considerations

- Passport images transmitted as base64 over HTTPS; server-side processing only — no dummy data in production path
- Admin routes protected by JWT middleware; role-based UI (Admin / Manager)
- Driver authentication via Supabase with session isolation per device
- PII minimization roadmap: tokenized passport references post-UAE API integration
- Audit trail: booking status transitions, slot_bookings timestamps, vehicle_tracking coordinates
- Alignment with UAE data residency requirements for production deployment on approved cloud regions

API Surface Summary

Method	Endpoint	Purpose
POST	/api/bookings/verify	Validate booking ID + airline
PUT	/api/bookings/:id/passport	Save verified passport fields
POST	/api/passport/scan	OCR + extract + optional booking match
GET	/api/slots/available	List bookable pickup windows
POST	/api/slots/book	Reserve slot + assign vehicle
POST	/api/slots/confirm-pickup	OTP verify + proof QR
GET	/api/otp/driver-tasks/:driverId	Driver pending/active tasks
POST	/api/otp/accept-booking	Driver accepts trip
GET	/api/admin/operations/*	Stats, vehicles, passengers, analytics
WS	Socket.IO	location_update, status_update, join_booking, join_dispatch
POST	AI /chat (port 8000)	Streaming bilingual assistant

Demo & Deployment Readiness

Component	Run command	Port
Backend API	cd backend && node server.js	5000
AI service	cd ai-service && uvicorn main:app	8000

Admin dashboard	cd admin-dashboard && npm run dev	5173
Passenger app	cd mobile-app && npx expo start	Expo
Driver app	cd driver-app && npx expo start	Expo

Closing — Request for DFF Review

City Terminal demonstrates a **complete vertical slice** of Dubai’s 10X ambition: proactive government-grade services, AI-first citizen experience, and real-time operations — not a mock-up, but a working multi-app ecosystem ready for pilot hardening and official API partnerships.

We welcome a technical walkthrough, UX review, and alignment session on prioritising **UAE Passport Verification API**, **luggage vision checks**, and **flight disruption notifications** for the next 10X milestone.